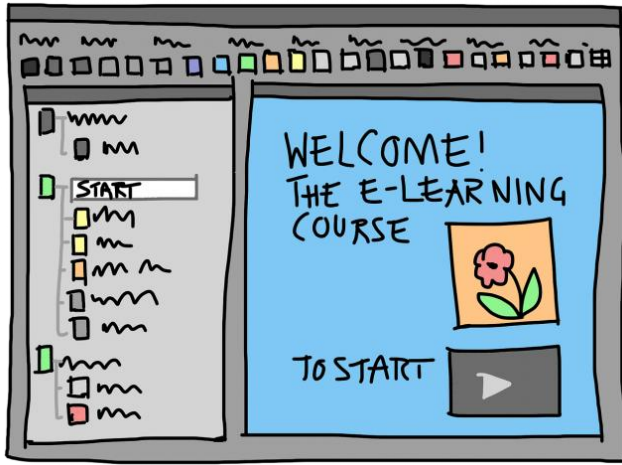




ENTORNOS
DE
DESARROLLO

REPOSITORIOS DE
SOFTWARE



Qué entregar

Realiza en un PDF las capturas de pantalla de todo el proceso.

1. Repositorios

Los repositorios de software son fundamentales para los programadores por varias razones que afectan directamente su eficiencia, colaboración y la calidad de su trabajo. Se encuentran integrados en numerosos IDE, de forma que se facilite su uso durante el desarrollo de aplicaciones.

Un repositorio de software es una plataforma o una base de datos en la que se almacenan y gestionan versiones de código fuente, bibliotecas, y otros recursos relacionados con el trabajo de un desarrollador. Tienen varios propósitos:

1. **Control de versiones**: Permiten llevar un registro de las diferentes versiones de un proyecto, lo que facilita el seguimiento de cambios, la colaboración entre desarrolladores y la reversión a versiones anteriores si es necesario.
2. **Colaboración**: Facilitan el trabajo en equipo, ya que varios desarrolladores pueden contribuir al mismo proyecto de manera simultánea. Los repositorios proporcionan herramientas para gestionar conflictos y fusionar cambios de manera eficiente.
3. **Distribución**: Ayudan en la distribución del software, permitiendo a los usuarios finales descargar e instalar aplicaciones y bibliotecas de manera sencilla.
4. **Seguridad**: Los repositorios suelen incluir características de seguridad, como la capacidad de firmar digitalmente el código, lo que ayuda a garantizar la autenticidad e integridad del software.
5. **Automatización**: Permiten la automatización de procesos, como la integración continua y la implementación continua (CI/CD), que son prácticas comunes en el desarrollo moderno de software.

Algunos ejemplos de plataformas de repositorios de software incluyen GitHub, GitLab y Bitbucket. Estos ofrecen servicios tanto para proyectos de código abierto como para proyectos privados, y son ampliamente utilizados en la comunidad de desarrollo de software.

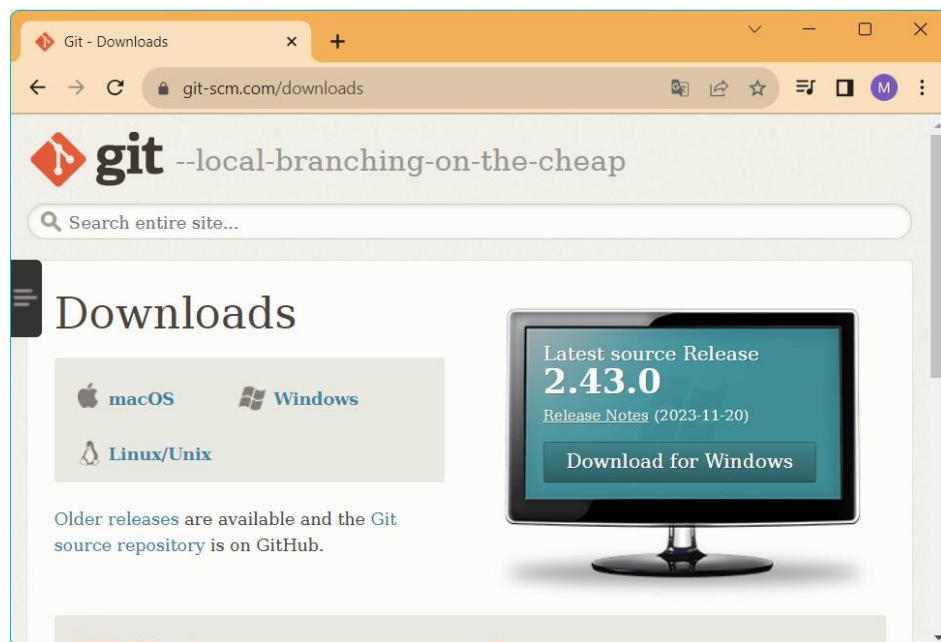


2. Git

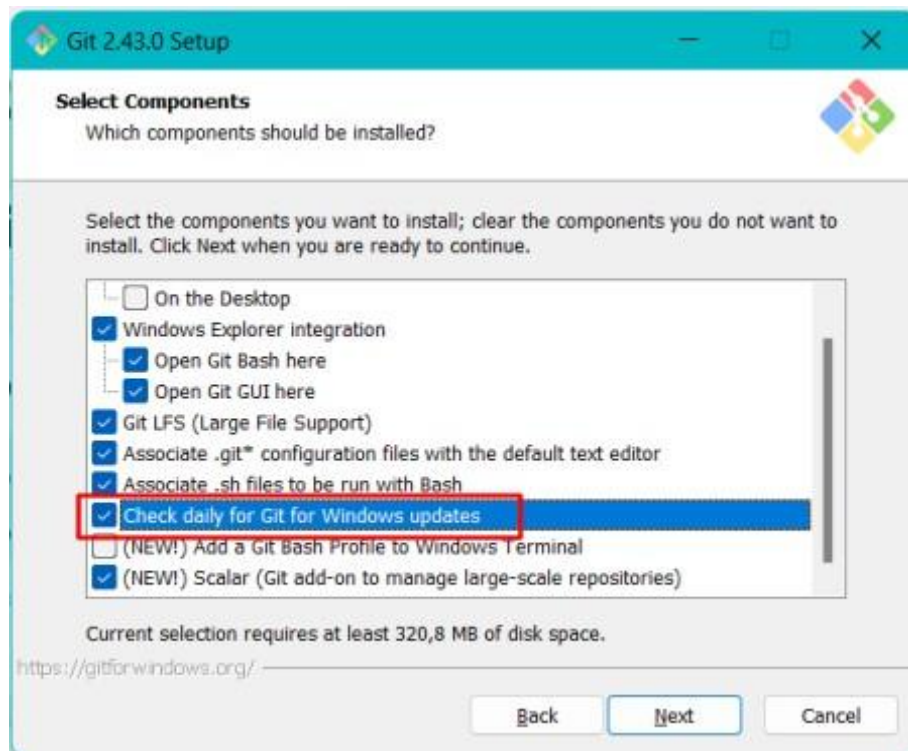
Se trata de un sistema de control de versiones distribuido y de código abierto que se utiliza de forma bastante generalizada en el desarrollo de software. Fue creado por Linus Torvalds en 2005 y se ha convertido en una herramienta fundamental para los desarrolladores. Con Git, cada desarrollador tiene una copia completa del historial de desarrollo, permitiendo el trabajo sin conexión y facilitando la colaboración en equipos distribuidos. Además, facilita la gestión de múltiples líneas de desarrollo de manera concurrente, algo esencial para proyectos grandes y complejos.

2.1. Instalación

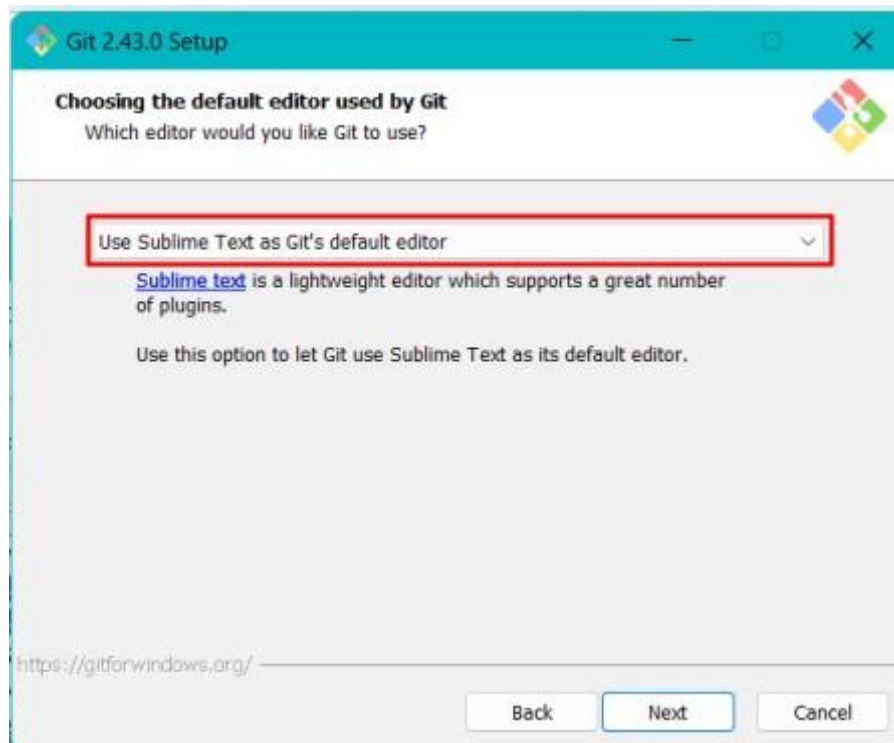
La mayoría de los sistemas operativos basados en Unix ya incluyen una versión de Git. Sin embargo, en los Sistemas Operativos Windows, deberás proceder a su descarga e instalación desde la página oficial <https://git-scm.com/downloads>.



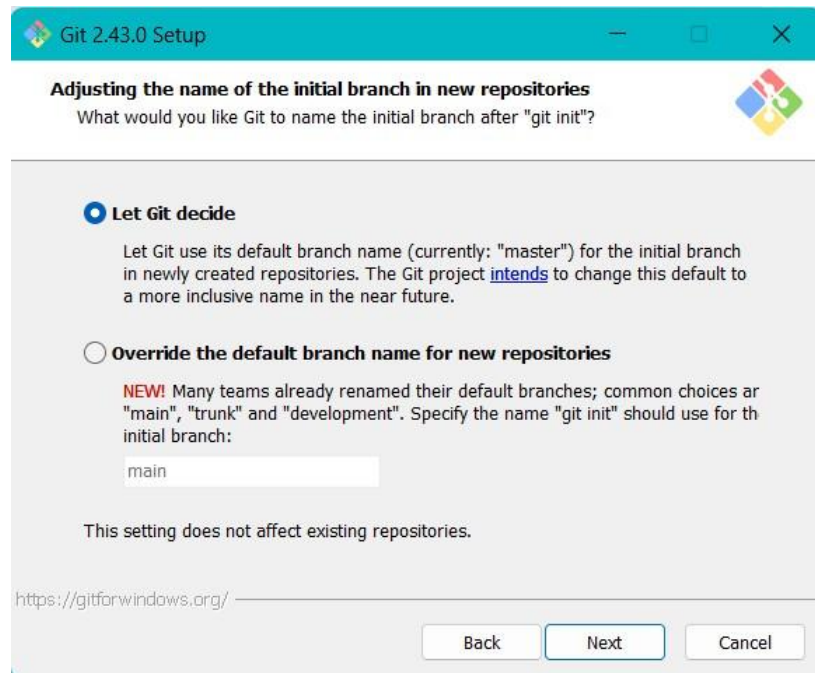
Y tras la descarga, lanza el ejecutable. Durante la instalación, comprueba que las actualizaciones diarias estén activadas:



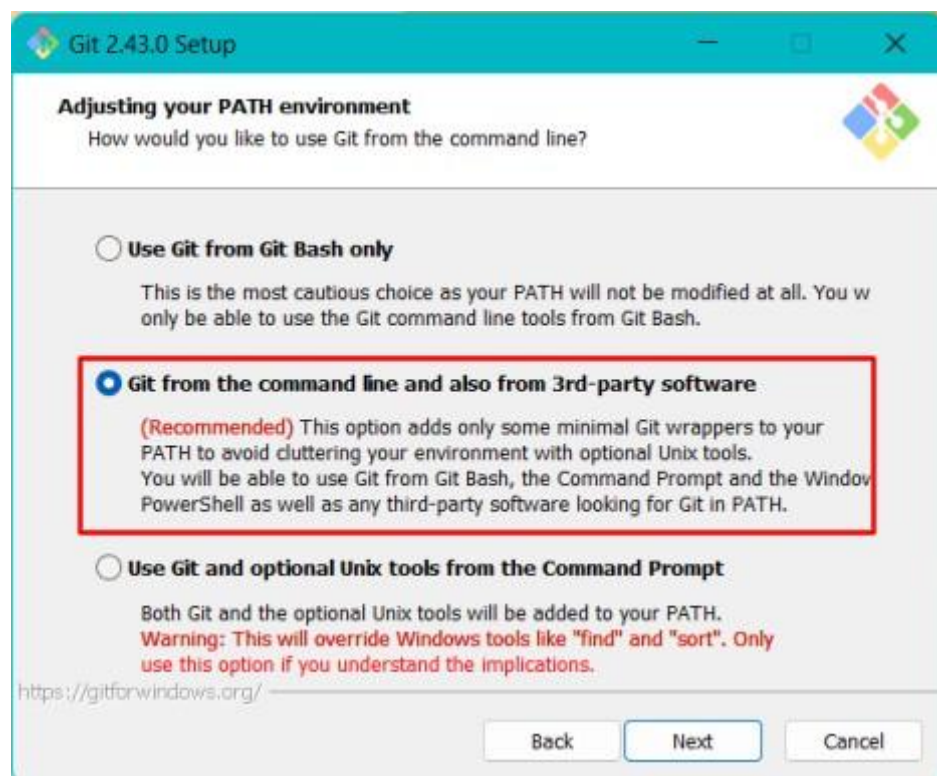
Y también establece un editor por defecto para Git. Escoge uno que sepas utilizar.



Deja que Git se encargue de nombrar la “rama inicial” (opción por defecto).



Y (muy importante), permite que se pueda usar Git tanto por línea de comandos como por otro software:



Continúa con el resto de opciones por defecto, e instálalo.



2.2. La terminal de Git

Ya puedes probar a abrir Git. Aparecerá una terminal, y si escribes “git”, aparecerán las diferentes opciones:

```
MINGW64; c/Users/violi
vio|i@LAPTOP-0P3TF93I MINGW64 ~
$ git
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
[--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
[-p | --paginate | -P | --no-pager] [--no-replace-objects] [--bare]
[--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
[--config-env=<name>=<envvar>] <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
  bisect    Use binary search to find the commit that introduced a bug
  diff      Show changes between commits, commit and working tree, etc
```

Las más usadas son las siguientes:

1. **git init**: para inicializar un nuevo repositorio Git en el directorio actual.
2. **git clone**: Clona un repositorio Git existente en un nuevo directorio.
3. **git add**: Añade cambios al área de preparación (staging) para el próximo commit.
4. **git commit**: Guarda los cambios en el área de preparación.
5. **git status**: Muestra el estado actual del repositorio, incluyendo archivos sin rastrear y cambios pendientes.
6. **git pull**: Obtiene y fusiona los cambios desde un repositorio remoto.
7. **git push**: Sube cambios locales a un repositorio remoto.
8. **git branch**: Muestra las ramas existentes y crea nuevas ramas.
9. **git checkout**: Cambia entre ramas o crea y cambia a una nueva rama.
10. **git merge**: Fusiona una rama con la rama actual.
11. **git log**: Muestra el historial de commits.



12. **git diff**: Muestra las diferencias entre los cambios en el área de trabajo y el último commit.

2.3. Repositorios

Un repositorio en Git es un espacio de almacenamiento que contiene el historial completo de un proyecto, incluyendo todos los archivos, carpetas y la información sobre cada cambio realizado en dicho proyecto. Almacena todas las versiones del código fuente, así como información sobre las ramas, commits, etiquetas, etc. Este almacenamiento puede ser local (en tu máquina), remoto (en un servidor, como GitHub), o mixto.

2.4. Commit

Se trata de un comando utilizado para registrar cambios en el repositorio. Cada commit representa un conjunto de cambios realizados en los archivos del proyecto en un momento específico. Los commits son fundamentales en el control de versiones, ya que permiten rastrear la evolución del código a lo largo del tiempo.

Cuando realizas un commit en Git, estás guardando una instantánea (*snapshot*) del estado actual de tu proyecto en ese momento. Cada commit tiene una identificación única (*hash*) que lo diferencia de otros commits y contiene la información sobre quién hizo el cambio, cuándo se hizo y un mensaje descriptivo que resume los cambios realizados.

Los commits son esenciales para realizar un seguimiento preciso de la evolución del código, facilitar la colaboración en equipo y revertir cambios si es necesario.

2.5. Ramas

Es posible que en un momento concreto nos interese hacer cambios en nuestro proyecto, o explorar una solución que no tenemos claro si podrá llevarse a cabo. En esos momentos nos interesa la creación de una ramificación, rama, o “branch”. Se trata de una línea independiente de desarrollo que permite realizar cambios en el código sin afectar la rama principal (generalmente llamada **master** o **main**), y facilita la implementación de nuevas características, correcciones de errores u otros cambios sin afectar directamente con la rama



principal. Cada rama tiene su propio historial de commits, pero todas comparten el mismo conjunto de archivos base del repositorio.

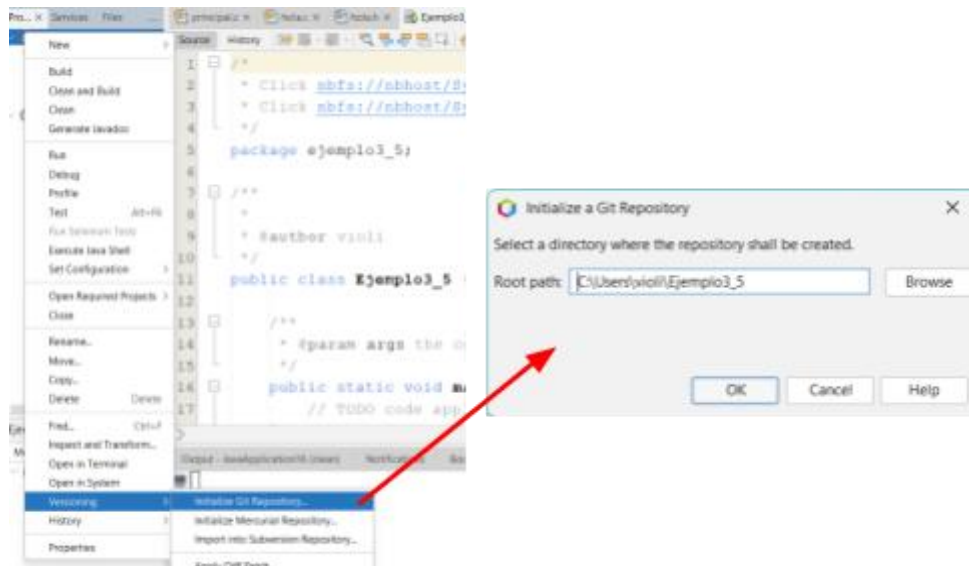
3. Git en NetBeans

Dependiendo de las acciones que queramos realizar, elegiremos unas u otras opciones dentro de la opción “Team” del menú de NetBeans:

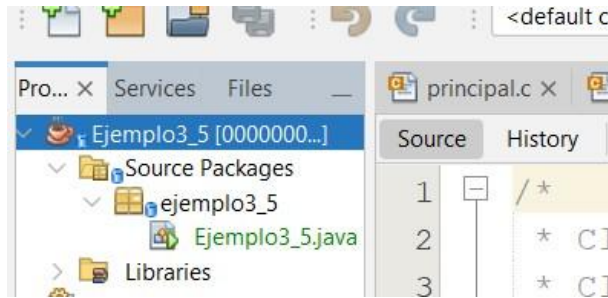
3.1. Inicializar un Repositorio:

Para crear nuestro repositorio en NetBeans:

1. **Abre o Crea un Proyecto.**
2. **Habilita Git en el Proyecto:** Si estás creando un nuevo proyecto, puedes habilitar la opción "Git" en el asistente de creación de proyectos, o esperar a que ya esté creado .
3. **Inicia el Repositorio:** Si estás trabajando en un proyecto existente, dirígete a "Team" en la barra de menú y selecciona "Initialize Git Repository", o pincha con el botón derecho, y dirígete a “Versioning->Initialize Git Repository”.



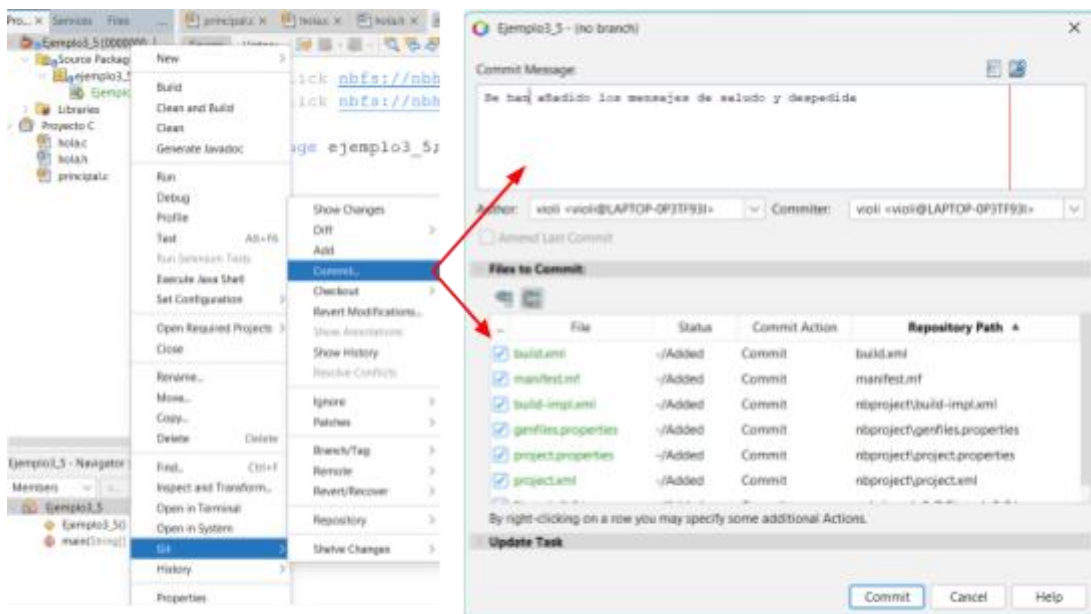
Verás cómo cambia la apariencia de tu proyecto:



3.2. Realizar Commits:

Supongamos que has realizado cambios en tu proyecto y quieres almacenarlos:

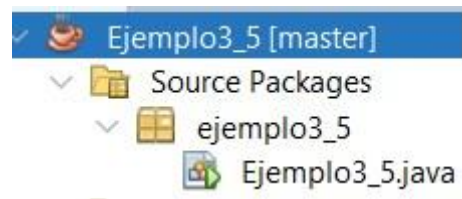
1. **Haz “commit”:** Puedes dirigirte a la opción "Team" en la barra de menú, o bien pinchar sobre el proyecto con el botón derecho. Después, selecciona "Git -> Commit".
2. **Selecciona los Archivos:** Marca las casillas de los archivos que deseas incluir en el commit.
3. **Agrega un Mensaje de Commit:** Debe ser descriptivo.



4. **Realiza el Commit:** Así confirmarás los cambios.

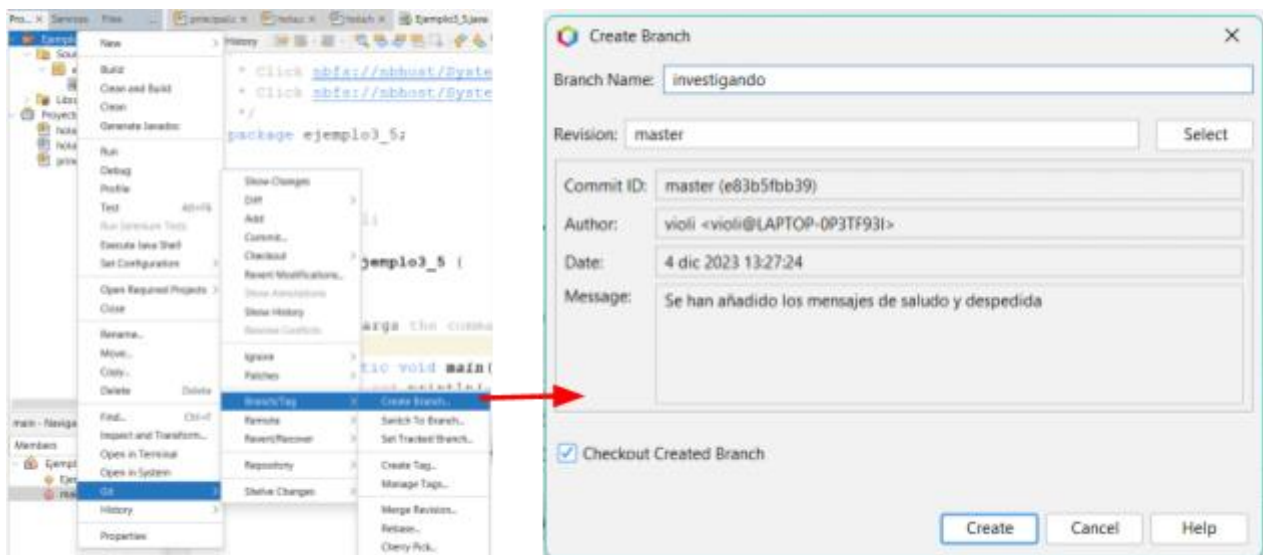
3.3. Trabajar con Ramas

Inicialmente, todo proyecto tiene una única rama, denominada **main** o **master**:



Pero como hemos comentado previamente, es posible que necesitemos crear una nueva rama. Para ello:

1. **Abrimos la ventana de Git Branches:** Desde "Team" en la barra de menú, o haciendo click con el botón derecho. Selecciona "Git->Branches".
2. **Creamos la Nueva Rama:** Haz clic en "Create Branch" e introduce el nombre de la nueva rama.



3. **Realiza Cambios en la Rama:** El proyecto habrá cambiado a la nueva rama, en la que podrás realizar los cambios que desees.
4. **Fusiona las Ramas:** Cuando hayas realizado las pruebas necesarias, quizá esté todo listo para fusionar los cambios. Para ello, vuelve a la rama principal, selecciona "Merge" desde la ventana de Git, y selecciona la rama que desees fusionar.